Navier-Stokes PINN Code Flow and Non-OOP PyTorch Rewrite

Based on the uploaded TensorFlow PINN script: NavierStokesOriginal.txt

Navier-Stokes PINN Code Flow and Non-OOP PyTorch Rewrite

Source file reviewed: NavierStokesOriginal.txt

Purpose of the original TensorFlow code

The uploaded script implements a Physics-Informed Neural Network (PINN) for the two-dimensional incompressible Navier-Stokes equations using cylinder-wake data. The code is not only predicting velocity and pressure; it is also learning two PDE coefficients:

```
lambda_1: coefficient multiplying the nonlinear convection terms
lambda_2: viscosity/diffusion coefficient
```

The intended governing form is:

```
u_t + lambda_1 (u u_x + v u_y) = -p_x + lambda_2 (u_xx + u_yy)
v_t + lambda_1 (u v_x + v v_y) = -p_y + lambda_2 (v_xx + v_yy)
```

For the reference data, the expected values are approximately lambda_1 = 1 and lambda_2 = 0.01.

Core PINN idea

The network takes three inputs:

```
x, y, t
```

and produces two outputs:

```
psi, p
```

Here, psi is a streamfunction and p is pressure. The code does not predict u and v directly. Instead, it defines velocity by differentiating the streamfunction:

```
u = d(psi)/dy
v = -d(psi)/dx
```

This is important because it automatically satisfies the incompressibility equation:

```
u_x + v_y = 0
```

The physics loss is then built by differentiating u, v, and p with respect to x, y, and t and substituting those derivatives into the Navier-Stokes momentum equations.

Flow of the original TensorFlow code

1. Imports and reproducibility

The script imports TensorFlow 1.x, NumPy, SciPy, matplotlib, grid interpolation tools, and custom plotting utilities. It fixes NumPy and TensorFlow random seeds.

2. PhysicsInformedNN.__init__

The constructor receives training columns x, y, t, u, and v. It concatenates x, y, and t into one input matrix X, then stores the lower and upper bounds of X. Those bounds are later used to normalize network inputs to roughly [-1, 1].

3. Network and trainable PDE parameters

The network is a fully connected tanh multilayer perceptron. The layer list in the script is:

```
[3, 20, 20, 20, 20, 20, 20, 20, 20, 2]
```

This means 3 inputs, eight hidden layers of width 20, and 2 outputs. The two output columns represent psi and p. The code also creates trainable TensorFlow variables lambda_1 and lambda_2, both initialized to zero.

4. TensorFlow placeholders and graph construction

Because the original code is TensorFlow 1.x, it uses placeholders for x, y, t, u, and v. It then builds a computation graph by calling net_NS, which returns predicted u, v, p, and the two residuals f_u and f_v.

5. Loss function

The loss has four parts:

```
velocity data loss for u
velocity data loss for v
PDE residual loss for f_u
PDE residual loss for f_v
```

In formula form:

```
loss = sum((u_observed - u_pred)^2)
     + sum((v_observed - v_pred)^2)
     + sum(f_u^2)
     + sum(f_v^2)
```

Pressure has no direct data-loss term in this script. Pressure is inferred indirectly through the PDE residuals.

6. Training strategy

Training happens in two stages. First, Adam is used for many iterations. Then SciPy L-BFGS-B is used to refine the solution. This two-stage approach is common in PINN examples: Adam gets close to a useful region, and L-BFGS often improves the final fit.

7. Prediction

The predict method feeds new x, y, and t arrays through the TensorFlow graph and returns u, v, and p predictions.

8. Main script data pipeline

The script loads cylinder_nektar_wake.mat. The original data shapes are:

```
U_star: N x 2 x T
p_star: N x T
t:      T x 1
X_star: N x 2
```

It tiles the spatial coordinates across all time points, tiles the time vector across all spatial points, flattens everything into NT x 1 columns, then randomly samples 5000 points for training.

9. Clean and noisy experiments

The model is first trained on clean velocity data. Then the script adds 1% Gaussian noise to u and v and trains again. The purpose is to test whether the PINN can still identify lambda_1 and lambda_2 when the observations are noisy.

10. Plotting

The final part of the file interpolates the predicted fields onto a regular grid and builds publication-style plots for vorticity, training data, pressure comparison, and a table of the identified PDE.

Important translation choices in the PyTorch rewrite

The rewrite below keeps the mathematical structure the same but removes the class-based design.

1. No custom class and no nn.Module

The network weights, biases, lambda_1, and lambda_2 are plain torch.nn.Parameter objects stored in Python lists and variables.

2. No TensorFlow placeholders or session

PyTorch uses eager execution. Each loss calculation rebuilds the derivative graph dynamically.

3. Autograd replaces tf.gradients

The helper function grad(...) wraps torch.autograd.grad(...). create_graph=True is required because the PDE residuals need second derivatives.

4. PyTorch LBFGS replaces SciPy L-BFGS-B

The TensorFlow code uses tf.contrib.opt.ScipyOptimizerInterface with L-BFGS-B. PyTorch has torch.optim.LBFGS. Since this problem does not use explicit parameter bounds, LBFGS is a reasonable replacement.

5. Prediction cannot use torch.no_grad()

The model predicts u and v by differentiating psi with respect to x and y, so autograd must remain enabled during prediction.

6. Long training time

The original script uses 200,000 Adam iterations plus up to 50,000 L-BFGS iterations. The rewrite keeps those defaults for faithfulness, but for a quick test you should reduce adam_iter and lbfgs_iter.

How to run the PyTorch version

Install the main dependencies:

```
pip install numpy scipy torch
```

Place the data file so the relative path matches the script:

```
../Data/cylinder_nektar_wake.mat
```

Then run:

```
python navier_stokes_pinn_pytorch_no_oop.py
```

For a smoke test, edit the final block and use smaller iteration counts, for example:

```
clean_results = run_experiment(noise=0.0, adam_iter=100, lbfgs_iter=20)
```

Complete non-OOP PyTorch rewrite

The full code is included below and also provided as a separate .py file.

Appendix: Full PyTorch code

This appendix is the same code as the separate Python file. It is intentionally written without a custom class or nn.Module.

```
"""
Non-OOP PyTorch rewrite of the TensorFlow PINN for 2D incompressible
Navier-Stokes parameter discovery.


The network maps (x, y, t) -> (psi, p), where psi is the streamfunction
and p is pressure. Velocity is recovered by differentiating psi:
    u = d psi / d y
    v = - d psi / d x
This automatically satisfies incompressibility: u_x + v_y = 0.


The trainable scalars lambda_1 and lambda_2 identify the coefficients in:
    u_t + lambda_1 * (u u_x + v u_y) = -p_x + lambda_2 * (u_xx + u_yy)
    v_t + lambda_1 * (u v_x + v v_y) = -p_y + lambda_2 * (v_xx + v_yy)


Expected data file:
    ../Data/cylinder_nektar_wake.mat
with keys: U_star, p_star, t, X_star.
"""


import time
from typing import Dict, List, Sequence, Tuple


import numpy as np
import scipy.io
import torch
from scipy.interpolate import griddata



def set_seed(seed: int = 1234) -> None:
    """Make NumPy and PyTorch runs more reproducible."""
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)



def make_tensor(array: np.ndarray, device: torch.device, requires_grad: bool = False) -> torch.Tensor:
    """Convert a NumPy array to a float32 tensor on the chosen device."""
    tensor = torch.as_tensor(array, dtype=torch.float32, device=device)
    tensor.requires_grad_(requires_grad)
    return tensor



def initialize_parameters(layers: Sequence[int], device: torch.device) -> Tuple[List[torch.nn.Parameter], List[torch.nn.Parameter]]:
    """
    Create trainable weights and biases for a fully connected tanh network.


    This replaces the TensorFlow class method initialize_NN. The return values
    are ordinary Python lists, not an nn.Module subclass.
    """
    weights: List[torch.nn.Parameter] = []
    biases: List[torch.nn.Parameter] = []


    for in_dim, out_dim in zip(layers[:-1], layers[1:]):
        weight = torch.empty(in_dim, out_dim, dtype=torch.float32, device=device)
```

```
        torch.nn.init.xavier_normal_(weight)
        bias = torch.zeros(1, out_dim, dtype=torch.float32, device=device)

        weights.append(torch.nn.Parameter(weight))
        biases.append(torch.nn.Parameter(bias))

    return weights, biases



def neural_net(
    X: torch.Tensor,
    weights: Sequence[torch.nn.Parameter],
    biases: Sequence[torch.nn.Parameter],
    lb: torch.Tensor,
    ub: torch.Tensor,
) -> torch.Tensor:
    """
    Forward pass for the PINN.

    Inputs are scaled from their physical ranges to approximately [-1, 1],
    matching the TensorFlow implementation.
    """
    H = 2.0 * (X - lb) / (ub - lb) - 1.0

    for W, b in zip(weights[:-1], biases[:-1]):
        H = torch.tanh(H @ W + b)

    return H @ weights[-1] + biases[-1]



def grad(output: torch.Tensor, input_: torch.Tensor) -> torch.Tensor:
    """
    Compute d(output)/d(input_) while keeping the graph for higher derivatives.

    create_graph=True is required because the Navier-Stokes residual needs
    second derivatives such as u_xx and v_yy.
    """
    return torch.autograd.grad(
        output,
        input_,
        grad_outputs=torch.ones_like(output),
        create_graph=True,
        retain_graph=True,
    )[0]



def net_ns(
    x: torch.Tensor,
    y: torch.Tensor,
    t: torch.Tensor,
    weights: Sequence[torch.nn.Parameter],
    biases: Sequence[torch.nn.Parameter],
    lb: torch.Tensor,
    ub: torch.Tensor,
    lambda_1: torch.nn.Parameter,
    lambda_2: torch.nn.Parameter,
) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]:
    """
    Compute velocity, pressure, and PDE residuals.

    The network output has two columns:
        column 0: streamfunction psi
```

```
        column 1: pressure p
    """
    psi_and_p = neural_net(torch.cat([x, y, t], dim=1), weights, biases, lb, ub)
    psi = psi_and_p[:, 0:1]
    p = psi_and_p[:, 1:2]


    # Streamfunction representation. This enforces u_x + v_y = 0 exactly.
    u = grad(psi, y)
    v = -grad(psi, x)


    # First and second derivatives for the momentum equations.
    u_t = grad(u, t)
    u_x = grad(u, x)
    u_y = grad(u, y)
    u_xx = grad(u_x, x)
    u_yy = grad(u_y, y)


    v_t = grad(v, t)
    v_x = grad(v, x)
    v_y = grad(v, y)
    v_xx = grad(v_x, x)
    v_yy = grad(v_y, y)


    p_x = grad(p, x)
    p_y = grad(p, y)


    f_u = u_t + lambda_1 * (u * u_x + v * u_y) + p_x - lambda_2 * (u_xx + u_yy)
    f_v = v_t + lambda_1 * (u * v_x + v * v_y) + p_y - lambda_2 * (v_xx + v_yy)


    return u, v, p, f_u, f_v



def loss_fn(
    x_data: torch.Tensor,
    y_data: torch.Tensor,
    t_data: torch.Tensor,
    u_data: torch.Tensor,
    v_data: torch.Tensor,
    weights: Sequence[torch.nn.Parameter],
    biases: Sequence[torch.nn.Parameter],
    lb: torch.Tensor,
    ub: torch.Tensor,
    lambda_1: torch.nn.Parameter,
    lambda_2: torch.nn.Parameter,
) -> torch.Tensor:
    """
    PINN loss = data mismatch + physics residual mismatch.


    Fresh differentiable copies of x, y, and t are made every call so that
    PyTorch can rebuild the derivative graph cleanly during Adam and LBFGS.
    """
    x = x_data.detach().clone().requires_grad_(True)
    y = y_data.detach().clone().requires_grad_(True)
    t = t_data.detach().clone().requires_grad_(True)


    u_pred, v_pred, _, f_u_pred, f_v_pred = net_ns(
        x, y, t, weights, biases, lb, ub, lambda_1, lambda_2
    )


    # Sums are used to mirror the original TensorFlow code.
    data_loss = torch.sum((u_data - u_pred) ** 2) + torch.sum((v_data - v_pred) ** 2)
    physics_loss = torch.sum(f_u_pred**2) + torch.sum(f_v_pred**2)
    return data_loss + physics_loss
```

```
def train_adam(
    x_train: torch.Tensor,
    y_train: torch.Tensor,
    t_train: torch.Tensor,
    u_train: torch.Tensor,
    v_train: torch.Tensor,
    weights: Sequence[torch.nn.Parameter],
    biases: Sequence[torch.nn.Parameter],
    lb: torch.Tensor,
    ub: torch.Tensor,
    lambda_1: torch.nn.Parameter,
    lambda_2: torch.nn.Parameter,
    n_iter: int = 200_000,
    lr: float = 1e-3,
    print_every: int = 10,
) -> None:
    """Run the Adam phase, similar to tf.train.AdamOptimizer in the original."""
    parameters = list(weights) + list(biases) + [lambda_1, lambda_2]
    optimizer = torch.optim.Adam(parameters, lr=lr)


    start_time = time.time()
    for it in range(n_iter):
        optimizer.zero_grad(set_to_none=True)
        loss = loss_fn(
            x_train, y_train, t_train, u_train, v_train,
            weights, biases, lb, ub, lambda_1, lambda_2,
        )
        loss.backward()
        optimizer.step()


        if it % print_every == 0:
            elapsed = time.time() - start_time
            print(
                f"It: {it}, Loss: {loss.item():.3e}, "
                f"l1: {lambda_1.item():.3f}, l2: {lambda_2.item():.5f}, "
                f"Time: {elapsed:.2f}"
            )
            start_time = time.time()



def train_lbfgs(
    x_train: torch.Tensor,
    y_train: torch.Tensor,
    t_train: torch.Tensor,
    u_train: torch.Tensor,
    v_train: torch.Tensor,
    weights: Sequence[torch.nn.Parameter],
    biases: Sequence[torch.nn.Parameter],
    lb: torch.Tensor,
    ub: torch.Tensor,
    lambda_1: torch.nn.Parameter,
    lambda_2: torch.nn.Parameter,
    max_iter: int = 50_000,
    history_size: int = 50,
) -> None:
    """
    Run the LBFGS phase.


    TensorFlow used scipy.optimize L-BFGS-B. PyTorch provides LBFGS rather
    than L-BFGS-B, but here there are no explicit parameter bounds, so LBFGS
    is the natural replacement.
    """
    parameters = list(weights) + list(biases) + [lambda_1, lambda_2]
    optimizer = torch.optim.LBFGS(
        parameters,
```

```
        max_iter=max_iter,
        history_size=history_size,
        line_search_fn="strong_wolfe",
        tolerance_grad=1e-9,
        tolerance_change=np.finfo(float).eps,
    )


    def closure() -> torch.Tensor:
        optimizer.zero_grad(set_to_none=True)
        loss = loss_fn(
            x_train, y_train, t_train, u_train, v_train,
            weights, biases, lb, ub, lambda_1, lambda_2,
        )
        loss.backward()
        print(
            f"Loss: {loss.item():.3e}, "
            f"l1: {lambda_1.item():.3f}, l2: {lambda_2.item():.5f}"
        )
        return loss


    optimizer.step(closure)



def predict(
    x_star_np: np.ndarray,
    y_star_np: np.ndarray,
    t_star_np: np.ndarray,
    weights: Sequence[torch.nn.Parameter],
    biases: Sequence[torch.nn.Parameter],
    lb: torch.Tensor,
    ub: torch.Tensor,
    lambda_1: torch.nn.Parameter,
    lambda_2: torch.nn.Parameter,
    device: torch.device,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Predict u, v, and p at new points."""
    x_star = make_tensor(x_star_np, device, requires_grad=True)
    y_star = make_tensor(y_star_np, device, requires_grad=True)
    t_star = make_tensor(t_star_np, device, requires_grad=True)


    u_pred, v_pred, p_pred, _, _ = net_ns(
        x_star, y_star, t_star, weights, biases, lb, ub, lambda_1, lambda_2
    )


    return (
        u_pred.detach().cpu().numpy(),
        v_pred.detach().cpu().numpy(),
        p_pred.detach().cpu().numpy(),
    )



def load_cylinder_wake_data(
    mat_path: str,
    n_train: int = 5_000,
    noise: float = 0.0,
) -> Tuple[Dict[str, np.ndarray], Dict[str, np.ndarray]]:
    """
    Load and rearrange the cylinder wake data exactly like the TensorFlow code.


    Returns:
        train: sampled training arrays x, y, t, u, v, and X
        full: full arrays needed for prediction and error calculations
    """
    data = scipy.io.loadmat(mat_path)
```

```
    U_star = data["U_star"]    # N x 2 x T
    P_star = data["p_star"]    # N x T
    t_star = data["t"]         # T x 1
    X_star = data["X_star"]    # N x 2


    N = X_star.shape[0]
    T = t_star.shape[0]


    # Repeat spatial coordinates across time, and time coordinates across space.
    XX = np.tile(X_star[:, 0:1], (1, T))
    YY = np.tile(X_star[:, 1:2], (1, T))
    TT = np.tile(t_star, (1, N)).T


    UU = U_star[:, 0, :]
    VV = U_star[:, 1, :]
    PP = P_star


    # Flatten to NT x 1 column vectors.
    x = XX.flatten()[:, None]
    y = YY.flatten()[:, None]
    t = TT.flatten()[:, None]
    u = UU.flatten()[:, None]
    v = VV.flatten()[:, None]
    p = PP.flatten()[:, None]


    idx = np.random.choice(N * T, n_train, replace=False)
    x_train = x[idx, :]
    y_train = y[idx, :]
    t_train = t[idx, :]
    u_train = u[idx, :]
    v_train = v[idx, :]


    if noise > 0.0:
        u_train = u_train + noise * np.std(u_train) * np.random.randn(*u_train.shape)
        v_train = v_train + noise * np.std(v_train) * np.random.randn(*v_train.shape)


    train = {
        "x": x_train,
        "y": y_train,
        "t": t_train,
        "u": u_train,
        "v": v_train,
        "X": np.concatenate([x_train, y_train, t_train], axis=1),
    }
    full = {
        "X_star": X_star,
        "U_star": U_star,
        "P_star": P_star,
        "t_star": t_star,
        "TT": TT,
        "x": x,
        "y": y,
        "t": t,
        "u": u,
        "v": v,
        "p": p,
    }
    return train, full



def make_model_parts(
    layers: Sequence[int],
    train_X: np.ndarray,
```

```
    device: torch.device,
) -> Tuple[List[torch.nn.Parameter], List[torch.nn.Parameter], torch.Tensor, torch.Tensor, torch.nn.Parameter, torch.nn.Parameter]:
    """Create network parameters, data-scaling bounds, and PDE coefficients."""
    weights, biases = initialize_parameters(layers, device)


    lb = make_tensor(train_X.min(axis=0, keepdims=True), device)
    ub = make_tensor(train_X.max(axis=0, keepdims=True), device)


    lambda_1 = torch.nn.Parameter(torch.tensor([0.0], dtype=torch.float32, device=device))
    lambda_2 = torch.nn.Parameter(torch.tensor([0.0], dtype=torch.float32, device=device))


    return weights, biases, lb, ub, lambda_1, lambda_2



def run_experiment(
    mat_path: str = "../Data/cylinder_nektar_wake.mat",
    n_train: int = 5_000,
    layers: Sequence[int] = (3, 20, 20, 20, 20, 20, 20, 20, 20, 2),
    adam_iter: int = 200_000,
    lbfgs_iter: int = 50_000,
    noise: float = 0.0,
) -> Dict[str, float]:
    """Train one clean or noisy model and evaluate it on snapshot 100."""
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    train, full = load_cylinder_wake_data(mat_path, n_train=n_train, noise=noise)


    x_train = make_tensor(train["x"], device)
    y_train = make_tensor(train["y"], device)
    t_train = make_tensor(train["t"], device)
    u_train = make_tensor(train["u"], device)
    v_train = make_tensor(train["v"], device)


    weights, biases, lb, ub, lambda_1, lambda_2 = make_model_parts(
        layers, train["X"], device
    )


    train_adam(
        x_train, y_train, t_train, u_train, v_train,
        weights, biases, lb, ub, lambda_1, lambda_2,
        n_iter=adam_iter,
    )
    train_lbfgs(
        x_train, y_train, t_train, u_train, v_train,
        weights, biases, lb, ub, lambda_1, lambda_2,
        max_iter=lbfgs_iter,
    )


    # Match the original test snapshot.
    snap = np.array([100])
    X_star = full["X_star"]
    U_star = full["U_star"]
    P_star = full["P_star"]
    TT = full["TT"]


    x_star = X_star[:, 0:1]
    y_star = X_star[:, 1:2]
    t_star = TT[:, snap]


    u_star = U_star[:, 0, snap]
    v_star = U_star[:, 1, snap]
    p_star = P_star[:, snap]


    u_pred, v_pred, p_pred = predict(
```

```
        x_star, y_star, t_star,
        weights, biases, lb, ub, lambda_1, lambda_2, device,
    )


    error_u = np.linalg.norm(u_star - u_pred, 2) / np.linalg.norm(u_star, 2)
    error_v = np.linalg.norm(v_star - v_pred, 2) / np.linalg.norm(v_star, 2)
    error_p = np.linalg.norm(p_star - p_pred, 2) / np.linalg.norm(p_star, 2)
    error_lambda_1 = abs(lambda_1.item() - 1.0) * 100.0
    error_lambda_2 = abs(lambda_2.item() - 0.01) / 0.01 * 100.0


    print(f"Error u: {error_u:e}")
    print(f"Error v: {error_v:e}")
    print(f"Error p: {error_p:e}")
    print(f"Error l1: {error_lambda_1:.5f}%")
    print(f"Error l2: {error_lambda_2:.5f}%")


    return {
        "error_u": float(error_u),
        "error_v": float(error_v),
        "error_p": float(error_p),
        "lambda_1": float(lambda_1.item()),
        "lambda_2": float(lambda_2.item()),
        "error_lambda_1_percent": float(error_lambda_1),
        "error_lambda_2_percent": float(error_lambda_2),
    }



def interpolate_for_plotting(
    X_star: np.ndarray,
    values: np.ndarray,
    grid_size: int = 200,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Optional helper matching the original griddata plotting step."""
    lb = X_star.min(axis=0)
    ub = X_star.max(axis=0)
    x = np.linspace(lb[0], ub[0], grid_size)
    y = np.linspace(lb[1], ub[1], grid_size)
    X, Y = np.meshgrid(x, y)
    V = griddata(X_star, values.flatten(), (X, Y), method="cubic")
    return X, Y, V



if __name__ == "__main__":
    set_seed(1234)


    # Clean-data run. For a quick smoke test, reduce adam_iter and lbfgs_iter.
    clean_results = run_experiment(noise=0.0)
    print("Clean-data results:", clean_results)


    # Noisy-data run, matching the original 1% noise experiment.
    noisy_results = run_experiment(noise=0.01)
    print("Noisy-data results:", noisy_results)
```